

Model Quantization of BERT and LLaMA

Sara Allan

October 2, 2026

Abstract

Large language models and Transformer-based models can contain millions or billions of parameters, resulting in significant memory and computational requirements. Model quantization is a technique that reduces these requirements by representing model weights and other numerical values using fewer bits.

This report explains the concept of quantization and discusses different numerical representations including FP32, FP16, INT8, and INT4. It also examines how quantization can be applied to models such as BERT and LLaMA, including its advantages, limitations, and practical applications.

Contents

1	Introduction	2
2	What is Quantization?	2
3	Numerical Representations	3
3.1	FP32	3
3.2	FP16	3
3.3	INT8	3
3.4	INT4	3
4	Quantization of BERT	3
5	Quantization of LLaMA	4
6	Advantages of Quantization	4
7	Limitations of Quantization	4
8	Coding Example	5
9	Memory Comparison	5
10	Conclusion	5
11	References	6

1 Introduction

Large neural network models contain a large number of parameters. These parameters require memory to store and computational resources to process. As models become larger, deploying them on devices with limited memory becomes increasingly difficult.

BERT is a Transformer-based model commonly used for natural language processing tasks such as text classification, question answering, and named entity recognition. LLaMA is a family of large language models designed for a variety of language-related tasks.

One approach for reducing the memory requirements of these models is **quantization**. Quantization reduces the numerical precision used to represent model parameters, allowing the model to require less memory.

2 What is Quantization?

Quantization is a technique that represents numerical values using fewer bits than the original representation. Neural network weights are commonly stored using floating-point formats such as FP32. Quantization can convert these values into lower-precision formats such as FP16, INT8, or INT4.

The approximate memory required to store model parameters can be expressed as:

$$M \approx N \times \frac{B}{8}$$

where:

- M is the approximate memory required in bytes,
- N is the number of model parameters,
- B is the number of bits used for each parameter.

Reducing the number of bits per parameter can therefore significantly reduce the memory required to store the model.

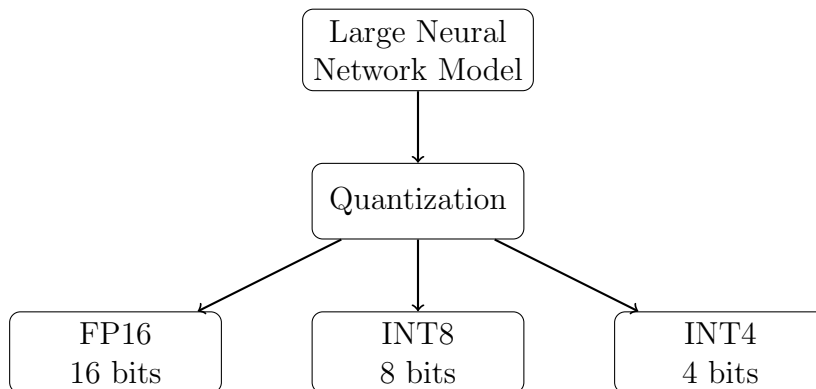


Figure 1: The basic concept of model quantization.

3 Numerical Representations

Different numerical representations use different numbers of bits.

Table 1: Comparison of numerical representations

Representation	Bits per value	Relative storage
FP32	32	100%
FP16	16	50%
INT8	8	25%
INT4	4	12.5%

3.1 FP32

FP32 uses 32 bits to represent each value and is commonly used as a standard high-precision representation for neural network parameters.

3.2 FP16

FP16 uses 16 bits per value. Compared with FP32, it can reduce the storage required for model weights by approximately half.

3.3 INT8

INT8 uses 8 bits to represent a value. In an idealized comparison, the raw storage required for the weights is approximately one quarter of FP32.

3.4 INT4

INT4 uses only 4 bits per value. This can provide further memory reduction compared with INT8, although lower precision can introduce greater quantization error.

4 Quantization of BERT

BERT-base contains approximately 110 million parameters. If its parameters were stored entirely using FP32, the approximate storage required for the weights would be:

$$110,000,000 \times \frac{32}{8} \approx 440 \text{ MB}$$

Using INT8 instead:

$$110,000,000 \times \frac{8}{8} \approx 110 \text{ MB}$$

This demonstrates how reducing the number of bits per parameter can reduce the storage requirements of a model.

5 Quantization of LLaMA

Quantization becomes particularly useful for large language models because they can contain billions of parameters.

For example, consider a model with 8 billion parameters. The approximate weight storage using FP32 is:

$$8,000,000,000 \times \frac{32}{8} = 32 \text{ GB}$$

Using INT8:

$$8,000,000,000 \times \frac{8}{8} = 8 \text{ GB}$$

Using INT4:

$$8,000,000,000 \times \frac{4}{8} = 4 \text{ GB}$$

These calculations demonstrate the potential memory savings obtained by using lower-precision representations.

6 Advantages of Quantization

The main advantages of quantization include:

- Reduced memory requirements.
- Smaller model storage size.
- Potentially faster inference.
- Lower hardware requirements.
- Easier deployment on devices with limited memory.

7 Limitations of Quantization

Quantization also introduces some limitations:

- Lower precision can introduce numerical error.
- Aggressive quantization can reduce model quality.
- Different models may respond differently to quantization.
- Additional techniques may be required to preserve model performance at very low precision.

8 Coding Example

The Hugging Face Transformers library provides support for loading models using quantization configurations.

For example, an 8-bit configuration can be created using `BitsAndBytesConfig`:

```
from transformers import AutoModelForSequenceClassification
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_8bit=True
)

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    quantization_config=quantization_config,
    device_map="auto"
)
```

The parameter `load_in_8bit=True` specifies that the model should be loaded using 8-bit quantization.

A 4-bit configuration can similarly be specified using:

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True
)
```

9 Memory Comparison

For an 8-billion-parameter model, the theoretical weight-storage requirements can be compared as follows:

Table 2: Theoretical memory comparison for an 8B parameter model

Representation	Bits	Approximate weight storage
FP32	32	32 GB
FP16	16	16 GB
INT8	8	8 GB
INT4	4	4 GB

These values represent theoretical weight storage. Actual memory usage during model inference can be different because of additional model components, runtime requirements, and quantization overhead.

10 Conclusion

Quantization is an important technique for reducing the memory requirements of neural network models. By representing parameters using fewer bits, models such as BERT and LLaMA can require substantially less storage for their weights.

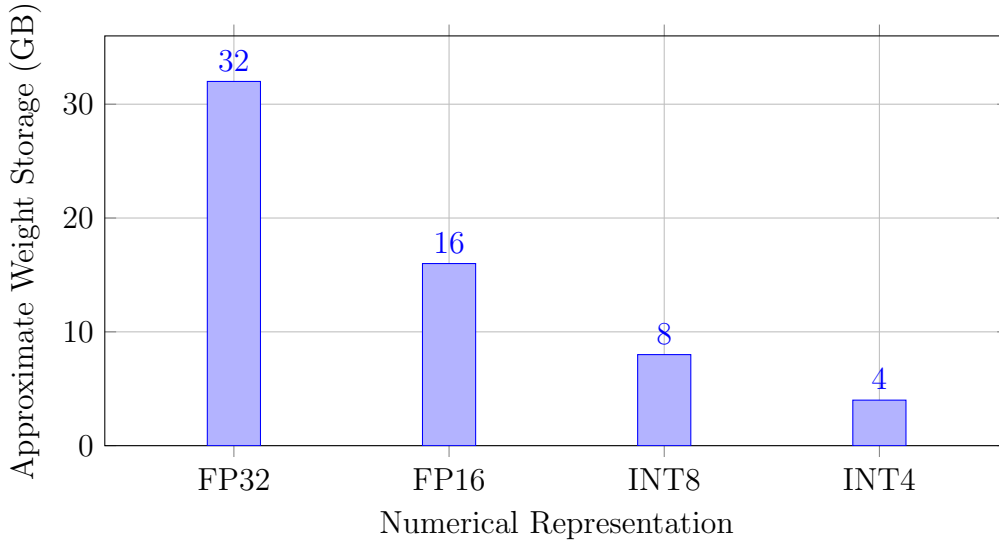


Figure 2: Theoretical weight-storage comparison for an 8-billion-parameter model.

FP16, INT8, and INT4 provide progressively lower numerical precision and can therefore provide increasing memory savings. However, lower precision may also introduce numerical errors and potentially affect model performance.

Therefore, quantization provides a practical trade-off between model size, computational requirements, and model quality.

11 References

1. Hugging Face, “Quantization,” Transformers Documentation. https://huggingface.co/docs/transformers/quantization/concept_guide
2. Hugging Face, “Bitsandbytes Quantization,” Transformers Documentation. <https://huggingface.co/docs/transformers/quantization/bitsandbytes>
3. Hugging Face, “Quantization,” Transformers Documentation. https://huggingface.co/docs/transformers/main_classes/quantization